

Jour 4 — Calculabilité

Machine de Turing & machine universelle

Modules : `turing.py`, `utm.py` • App : `apps/mtu` • Barème : 4 pts (+1)

Contexte

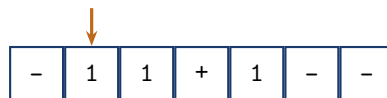
On lève la dernière limite : une mémoire **libre**, lisible *et* ré-inscriptible n'importe où — la machine de Turing. Puis une idée vertigineuse : **une seule** machine peut simuler **toutes** les autres si on lui donne leur description. C'est la machine **universelle**, ancêtre du processeur et de l'interpréteur.

Notions nouvelles du jour

1. La machine de Turing

$M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ avec $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$. Une **tête** lit/écrit une case et se déplace : « dans l'état q , je lis $a \rightarrow$ j'écris b , je bouge de d , je passe en q' ». Différence avec la pile (jour 2) : ici on peut **relire et réécrire** le passé.

tête, état q_0



*ruban bi-infini ;
blanc $_$; entrée
11+1.*

2. Une opération est une machine : trace de ADD sur 11+1

Table : $(q_0, 1) \rightarrow (q_0, 1, R)$; $(q_0, +) \rightarrow (q_1, 1, R)$; $(q_1, 1) \rightarrow (q_1, 1, R)$; $(q_1, _) \rightarrow (q_2, _, L)$; $(q_2, 1) \rightarrow (q_f, _, S)$. La case sous la tête est notée $[]$.

Trace pas-à-pas — $2+1 = 3$

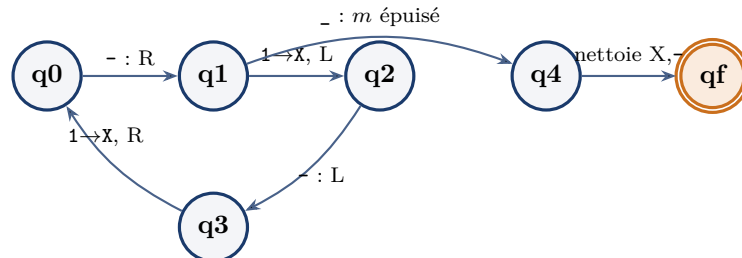
pas	état	ruban	action
0	q_0	$[1]1+1$	lit 1 $\rightarrow$ R
1	q_0	$1[1]+1$	lit 1 $\rightarrow$ R
2	q_0	$11[+]1$	le + devient 1, R, passe q_1
3	q_1	$111[1]$	lit 1 $\rightarrow$ R
4	q_1	$1111[_]$	blanc : demi-tour, passe q_2
5	q_2	$111[1]_$	efface le dernier 1, passe q_f
6	q_f	$111[_]$	accepté ; ruban = 111 = 3

Idée : $+ \rightarrow 1$ *fusionne* les deux blocs (on a $n+1+m$ uns), puis on retire **un 1**. Coût

$O(n+m)$.

3. Lire une table par *phases* : la soustraction

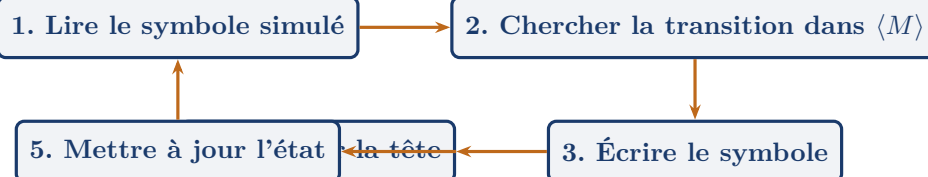
SUB apparie un 1 de m (marqué X) avec un 1 de n (barré X , de droite à gauche), jusqu'à épuiser m , puis nettoie. Chaque état est une **phase** :



Trois points délicats (à expliquer au rapport) : $(q0, X) \rightarrow R$ et $(q1, X) \rightarrow R$ — la table doit gérer ses *propres* marques ; le barrage **droite** $\rightarrow$ **gauche** donne une sortie contiguë ; la fin de m est détectée par un **blanc** (le ruban compte, pas un compteur).

4. Codage $\langle M \rangle$ et machine universelle

Une MT n'est qu'une **table finie** : on l'écrit comme une **chaîne** $\langle M \rangle$ (**injective**, **décodable**). Un *programme* devient une *donnée*. La machine **universelle** U prend $\langle M \rangle \#\#w$ et **simule** M (principe du *programme enregistré*). Un **cycle** :



Church–Turing & composition

« Toute fonction effectivement calculable est calculable par une MT » : pas un théorème, mais une thèse étayée par l'**équivalence** de modèles indépendants (λ -calcul, fonctions récursives, RAM). On obtient $\times$ et $\div$ par **composition** de MT (mult. = additions répétées) — ce que fait U en orchestrant des sous-programmes (apps/mtu). La calculatrice est **exécutée par** U (addition_via_utm).

5. La limite : indécidabilité

Revers de l'universalité : **on perd la décidabilité**. Le problème de l'**arrêt** (HALT) est indécidable (Turing 1936) ; par réduction, « U s'arrête sur $\langle M \rangle \#\#w$ » l'est aussi. *Universal* $\neq$ *omniscient* : U peut tout **simuler**, mais aucune machine ne **prédit** en général l'arrêt d'une autre.

A. Théorie (à rédiger) — 2 pts

Q4.1 (0,5)

Schéma d'encodage $\langle M \rangle$; justifier injectif + décodable.

Q4.2 (0,5)

Décrire U et son cycle de simulation (invariant + un tour).

Q4.3 (0,5)

Surcoût (**sourcé**) : multi-ruban $O(t \cdot |\langle M \rangle|)$; multi $\rightarrow$ mono $\leq$ quadratique; Hennie–Stearns (logarithmique).

Q4.4 (0,5)

Indécidabilité par réduction depuis HALT.

B. Pratique — 2 pts

E4.1 (0,5)

`TuringMachine.run` (ruban, arrêt, trace).

E4.2 (0,5)

`encode/decode` + `UniversalTM.run`.

E4.3 (0,75)

Tables ADD/SUB + Calculatrice ($\times, \div$ par composition).

E4.4 (0,25)

`UniversalInterpreter.run` (encode puis délègue à U).

Gate (règle non négociable)

`apps/mtu` ne réécrit aucune boucle de MT : il décrit des **tables** et les fait tourner via `formlang.turing/utm`. *Honnêteté* : surcoût **référéncé**, pas de constante « de mémoire ».

Références

Turing 1936. • Sipser (MTU, indécidabilité). • Hennie & Stearns 1966 (*J. ACM* 13(4)). • Arora & Barak 2009 (surcoûts).